Sprint 2 Meeting Prep: Guided Tours Implementation 10/13

1. Progress Update for Sponsors

New Files Implemented

UITour.vue — Tour Interface

- Displays the guided tour panel, including the header, search bar, and tour selection buttons.
- Handles the panel's styling, layout, colors, and scrolling.
- Provides four tour options:
    - Create a Point
    - Rotate Sphere
    - Create a Line Segment
    - Create a Circle
- Calls `selectTour()` in `UIFunctions.vue` when a user selects a tour.
- Passes search input to `searchTours()` in `UIFunctions.vue`.
- Uses Vue's `ref()` and a child component reference to access functions exposed by `UIFunctions.vue`.

UIFunctions.vue — Tour Logic

- Uses `defineExpose()` to make `selectTour()` and `searchTours()` available to `UITour.vue`.
- Defines the steps for each of the four tours, including target elements, popup titles, and instructional descriptions.
- Uses Driver.js to highlight interface elements and display step-by-step instructional popups.
- Implements `selectTour()` to find the selected tour, configure its steps, and start the tour.
- Uses CSS selectors to identify relevant tools, categories, and the sphere canvas in Spherical Easel.
- Includes a placeholder `searchTours()` function that logs the search text but does not yet filter the tour list.

Driver.js Features and Demo

- Demonstrate the current guided tours and their functionality.
- Discuss Driver.js features and potential applications.
- Identify opportunities to improve tour behavior through hooks and user action detection.

2. Discussion and Thoughts

What's Next?

We are one week ahead of schedule!

- Create additional guided tours.
- Refine the UI, including animated openings, grouping, and potential search bar improvements.
- Investigate user action detection to ensure users complete required actions before advancing.
- Investigate Firebase for long-term storage of completed tours and user progress.

Meeting Notes — October 13, 2026

1. Driver.js Function Hooks

- Investigate Driver.js hooks available through its configuration.
- Use hooks to check conditions on specific elements or at particular steps in a tour.
- Determine how to verify that users have completed the required actions before continuing.

2. Identifying UI Elements

- Use `git grep` to search the codebase for HTML elements and locate relevant buttons or components.
- Investigate adding unique identifiers to UI elements to make them easier to target.
- Use the following command to find existing test attributes:

git grep data-test

- The `data-` attribute prefix supports custom HTML attributes. For example, `data-tutid="someID"` could identify a specific UI element for a guided tour.

3. Detecting User Actions Through Pinia

- Dulimarta suggested checking the Pinia store to verify whether specific objects have been created at particular tour steps.
- Investigate checking the application's state to determine whether users have completed required actions.
- Use these checks to prevent users from advancing before completing the necessary steps.

Example: A button remains disabled until a point has been created.

4. Firebase Integration

- Investigate ways to store completed tours and user progress in Firebase.
- Determine how to persist tour completion data for long-term use.
- Explore how the application could retrieve and update this information.

5. More Advanced Guided Tours

- Advanced tours will require more complex validation and user action checks.
- Example: A tour that requires users to create two connected line segments and then measure the resulting segments.
- Determine how to validate relationships between created objects and confirm that the required calculations have been performed.

6. Long-Term Direction

- Focus future tours on teaching users to use the calculation feature in the Elements tab.
- Build tours that guide users through creating objects, connecting them, and performing calculations.
- Use application state and user action detection to make tours interactive rather than simply displaying instructions.